

DocuMind AI

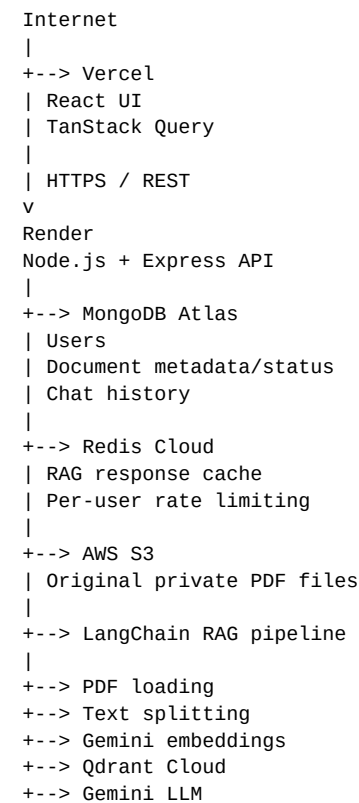
Intelligent Document Q&A with RAG

Complete Architecture, Data Flows, Request Flows & Interview Guide

Production stack: React + TanStack Query + Node.js/Express + MongoDB + Redis + AWS S3 + Gemini + LangChain + Qdrant

Deployment: Vercel (frontend) + Render (backend) + MongoDB Atlas + Qdrant Cloud + Redis Cloud + private AWS S3

1. High-Level Architecture

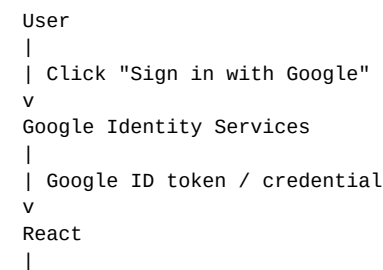


The backend is the orchestrator. MongoDB and Qdrant do not need to communicate directly: the Express application uses MongoDB for application metadata and Qdrant for vector retrieval.

2. Responsibilities of Each Technology

Technology	Responsibility
React	Frontend UI: authentication, upload, document list, chat and sources.
TanStack Query	Server-state fetching, caching, mutations and query invalidation.
Node.js + Express	REST API, authentication, authorization, orchestration and business logic.
MongoDB Atlas	Users, document metadata/status and persistent chat history.
Redis Cloud	RAG response caching and rate limiting.
AWS S3	Private durable storage for original PDFs.
LangChain	RAG pipeline orchestration, document loading/splitting, vector-store integration and LLM calls.
Gemini	Text embeddings and final grounded answer generation.
Qdrant Cloud	Vector storage and similarity search over document chunks.
Vercel	Production React hosting.
Render	Production Node/Express hosting.

3. Authentication Flow — Google + JWT



```

| POST /api/auth/google
| { credential }
v
Express auth controller
|
+--> Verify ID token with Google
|
+--> Find/create user in MongoDB
|
+--> Sign application JWT
|
v
React receives JWT + user
|
+--> localStorage.token
+--> localStorage.user
|
v
Protected API requests
Authorization: Bearer <JWT>

```

Google authentication uses an ID-token flow. The backend verifies the Google credential and then issues the application's own JWT. Protected routes read the JWT from the Authorization header.

4. Authorization Flow

```

Request
|
v
protect middleware
|
+-- Missing Bearer token --> 401
|
+-- Invalid/expired JWT --> 401
|
+-- Valid JWT
|
v
req.user.userId
|
v
Controller

```

Authorization is enforced server-side. Controllers use req.user.userId rather than trusting a user ID supplied by the browser.

5. Document Upload Flow

```

React
|
| multipart/form-data PDF
v
POST /api/documents/upload
|
v
JWT protect middleware
|
v
Multer memory storage
|
v
Document controller
|
+--> Generate unique S3 key
|
+--> Upload PDF to private S3
|
+--> Create MongoDB Document record
|
v

```

```

Document ingestion service
|
+--> Download PDF from S3
+--> Load PDF pages
+--> Split text into chunks
+--> Add documentId/userId/fileName/page metadata
+--> Create embeddings with Gemini
+--> Store vectors + metadata in Qdrant
|
v
MongoDB status = ready
|
v
API response
|
v
TanStack Query invalidates ["documents"]
|
v
Document list refreshes

```

6. RAG Ingestion Pipeline

```

PDF
|
v
PDF loader
|
v
Page documents
|
v
RecursiveCharacterTextSplitter
chunkSize ~1000
overlap ~200
|
v
Enriched chunks
documentId
userId
fileName
page
|
v
Gemini embedding model
|
v
Qdrant collection
documind_documents
|
+--> vector
+--> payload metadata

```

The metadata is important because retrieval is scoped by both `userId` and `documentId`. Qdrant payload indexes are created for `metadata.userId` and `metadata.documentId` so filtered similarity search works correctly.

7. Question / RAG Retrieval Flow

```

User asks question
|
v
POST /api/chat/ask
|
v
JWT protect
|
v
Redis rate limiter
|
+--> limit exceeded --> 429

```

```

|
v
Chat controller
|
v
RAG service
|
+--> Verify document belongs to user in MongoDB
|
+--> Build normalized question hash
|
+--> Redis cache lookup
|
+--> HIT --> return cached answer
|
+--> MISS
|
v
Qdrant similarity search
filtered by userId + documentId
|
v
Top relevant chunks
|
v
Build grounded prompt
|
v
Gemini LLM
|
v
Answer + source metadata
|
+--> Redis cache
|
+--> MongoDB chat history
|
v
Response to React

```

8. RAG Prompt / Grounding Strategy

The LLM is instructed to answer using only the retrieved document context. If the information cannot be found, it should respond that it could not find the information in the document. This reduces unsupported answers and keeps the application grounded in retrieved content.

The application returns source metadata including documentId, fileName and page so the UI can show where the answer came from.

9. Redis Caching Flow

```

Question
|
v
Normalize whitespace + lowercase
|
v
SHA-256(question)
|
v
Key:
rag:<userId>:<documentId>:<questionHash>
|
v
Redis GET
|
+--> HIT --> return stored { answer, sources }
|
+--> MISS

```

```

|
v
Run RAG
|
v
Redis SET
TTL = 3600 seconds
|
v
Return response

```

The cache key includes the user and document, preventing one user's cached answer from being reused for another user's document context.

10. Redis Rate-Limiting Flow

```

Protected request
|
v
Read req.user.userId
|
v
Redis INCR
key = rate-limit:<feature>:<userId>
|
+--> first request: set EXPIRE
|
+--> count <= limit --> next()
|
+--> count > limit --> HTTP 429

```

The current chat limiter is configured as an example policy of 20 requests per minute per authenticated user. Rate limits can be tuned without changing the core RAG pipeline.

11. Chat History Flow

```

Ask question
|
v
RAG answer
|
v
Find Chat by userId + documentId
|
+--> create if absent
|
+--> append user message
+--> append assistant message + sources
|
v
MongoDB
|
v
GET /api/chat/:documentId
|
v
React chat UI

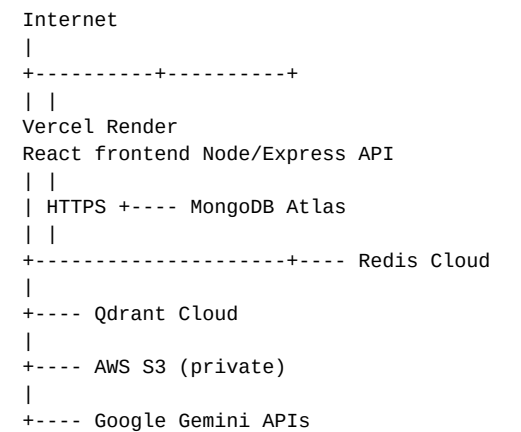
```

12. Data Ownership / Security Boundaries

Every protected document operation is scoped by the authenticated user ID. Document lookup checks both `_id` and `userId`. RAG retrieval applies Qdrant filters for `userId` and `documentId`. S3 stores original files privately. Secrets such as `JWT_SECRET`, Gemini keys, Qdrant API keys, Redis credentials and AWS secret credentials remain server-side.

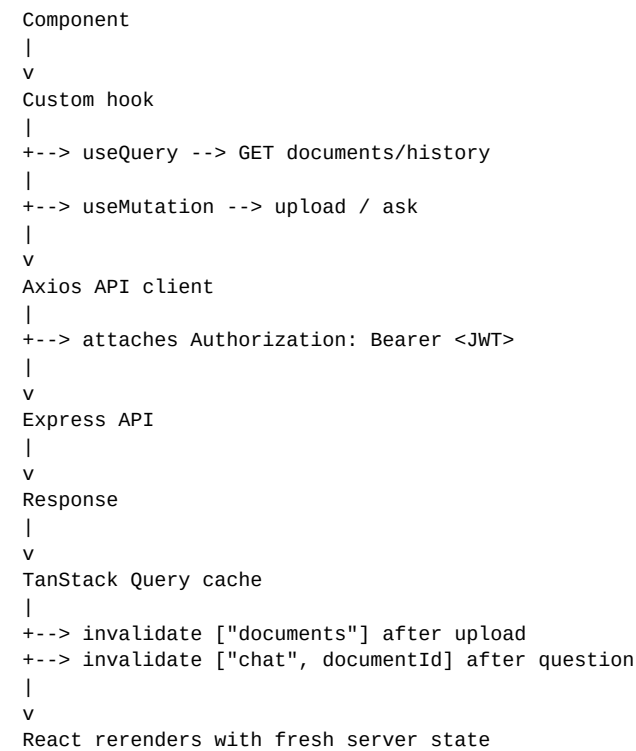
The frontend exposes only Vite variables intended for browser use, such as the Google client ID and API base URL. Backend-only secrets are never placed in the Vercel frontend environment.

13. Production Deployment Architecture



Local Docker Compose is used for development support services such as Qdrant/Redis. Production uses managed services. Unsecured local Qdrant/Redis ports should not be exposed to the public internet.

14. Frontend Request-State Flow with TanStack Query



This separates UI state from server state and avoids manually synchronizing document and chat data after mutations.

15. Important API Surface

Method	Endpoint	Purpose
POST	/api/auth/google	Verify Google credential and issue app JWT.
GET	/api/documents	Return authenticated user's documents.
POST	/api/documents/upload	Upload PDF, store in S3 and ingest into Qdrant.
POST	/api/chat/ask	Run authenticated, rate-limited RAG question answering.
GET	/api/chat/:documentId	Return chat history for a user's document.
GET	/api/health	Backend health check.

16. Project Structure

```
DocuMind-AI/  
  client/  
    src/  
      components/  
      hooks/  
      lib/  
      App.jsx  
      main.jsx  
      .env  
      .gitignore  
  server/  
    src/  
      config/  
      controllers/  
      middlewares/  
      models/  
      routes/  
      services/  
      app.js  
      .env  
      .gitignore  
  docker-compose.yml
```

17. Interview Explanation — 60 Seconds

“DocuMind AI is a production-style MERN application that lets authenticated users upload PDFs and ask questions about them. Google Identity Services handles login; my backend verifies the Google ID token and issues a JWT for protected APIs. PDFs are stored privately in S3, metadata and chat history are stored in MongoDB, and LangChain runs the RAG pipeline. During ingestion, PDF pages are split into chunks, enriched with document/user/page metadata, embedded with Gemini and stored in Qdrant. For a question, the backend verifies document ownership, checks Redis for a cached response, then performs filtered similarity search in Qdrant and sends the retrieved context to Gemini to generate a grounded answer with sources. Redis also provides per-user rate limiting. The frontend is deployed on Vercel and the backend on Render, with managed MongoDB, Redis and Qdrant services.”

18. Git Development Checkpoints

Confirmed history during development:

```
00b5dab secure document rag retrieval  
a1fa122 add jwt authentication  
52227f5 add persistent chat history  
de45301 preserve pdf page metadata in rag sources  
d418ff0 add redis caching for rag responses
```

Additional implementation work included S3 document upload, Qdrant/Redis configuration, RAG ingestion, frontend/TanStack Query integration and production deployment. A production start script was also added; whether that change has been committed should be verified with git status/log before claiming it as a checkpoint.

19. Operational / Production Notes

Keep MongoDB, Redis, Qdrant and S3 credentials in server-side environment variables. Keep S3 bucket private. Restrict production CORS to the deployed frontend origin. Keep Qdrant and Redis private/managed rather than exposing local development ports. Monitor Render logs for ingestion/RAG failures. The free Render instance may cold-start, so initial production requests can be slower.

A useful production test matrix is: Google login, protected API without JWT, expired/invalid JWT, PDF upload, non-PDF rejection, oversized PDF rejection, ingestion failure, document isolation between users, relevant RAG question, irrelevant question, repeated question for cache hit, rate-limit threshold, chat-history reload, mobile UI, and production CORS.